National University of Sciences & Technology

School of Electrical Engineering and Computer Science

Department of Computing

CS 417: Large Language Models (2+1)

Fall 2026

ASSIGNMENT 1

Building an Urdu Byte-Pair Encoding Tokenizer from Scratch

Assignment Report

Group Members

No.	Name	Registration No.
1	Ch Daniyal	453835
2	Malik Muhammad Hassaan	460233
3	Syed Danish Abbas	463657

Submitted to: Dr. Naima Iltaf

Submission: source code, four-page report, and separately shared large corpus files

Urdu BPE Tokenizer from Scratch

1. Overview

This report describes a byte-level BPE tokenizer written without a tokenizer library. It was trained on a new Urdu corpus assembled for this submission. The corpus uses Urdu Wikipedia, Urdu Wikisource and Urdu mC4, so it does not reuse Daily Express, Daily Jang, BBC Urdu, Makhzan or CC-100 from the supplied solution. The final vocabulary contains exactly 50,000 token IDs.

Design choice	Implementation
Base vocabulary	IDs 0-255 are raw byte values. This makes every Unicode string lossless and removes the need for an unknown token.
Special token	`<|endoftext|>` has ID 256 and never enters pair counting or merging.
Learned merges	49,743 ordered merges occupy IDs 257-49,999. A SHA-256 checksum protects their order in `tokenizer.json`.
Tie-breaking	The pair with greatest frequency is selected; tied pairs use the lexicographically greater integer pair.
Pre-tokenization	The Urdu-aware regex keeps combining marks and internal ZWNJ inside a word and attaches one leading space to the next unit.
Training	The reference trainer recounts all pairs. The fast trainer uses weighted unique types, a pair-to-word index, local count updates and an encoding cache.
Normalization	NFC normalization maps Arabic yeh, kaf and heh variants, uses ASCII digits, removes tatweel and controls, and preserves meaningful Urdu heh forms.

2. Corpus and Training Setup

The clean corpus contains **609.05 MiB**, 129,770 documents, 74,627,570 words and 857,782 unique word forms. The two Wikimedia collectors produced 118.34 MiB of raw self-collected text with per-page provenance records.

Source	Genre	Raw MiB	Clean MiB	Documents	Licence or terms
Urdu Wikipedia API	Encyclopedia	70.54	22.72	19,621	CC BY-SA 4.0 / GFDL
Urdu Wikisource API	Literature	47.80	43.41	11,921	CC BY-SA 4.0 + page notices
Urdu mC4 (AllenAI C4)	Diverse web	560.00	542.91	98,228	ODC-BY; underlying rights vary

Cleaning removes wiki and HTML markup, URLs, email addresses, phone-like strings and IP addresses. It applies the idempotent normalizer, filters low-Urdu lines, removes lines found in at least 50 documents, and removes exact normalized duplicates by SHA-256. A source-stratified shuffle with seed 4,172,026 produced a 98/1/1 document split. The three split hash sets are pairwise disjoint.

Item	Measured value
Training split	597.20 MiB; 127,173 documents; 73,175,999 words
Validation and test	5.91 MiB and 5.94 MiB
Reference versus fast check	500 identical merges on 1.00 MiB; 428.91 s versus 6.96 s
Full training	31.12 minutes; 8 logical CPUs; 15.7 GiB RAM; Python 3.11.0
Encoded output	88,404,583 train IDs and 876,941 validation IDs, stored as flat `uint16`

3. Question 1: What Did the Merges Learn?

Urdu letters usually occupy two UTF-8 bytes. Early merges therefore rebuild letters before they learn complete words. In the tables below, ␣ denotes a leading space and byte escapes denote intermediate tokens that are not valid UTF-8 by themselves.

Merges 1-50

#1 ا d8 a7	**#2** ی db 8c	**#3** \x20\xd8 20 d8	**#4** ک da a9	**#5** \x20\xd9 20 d9	**#6** ر d8 b1	**#7** و d9 88	**#8** ہ db 81	**#9** ے db 92	**#10** ␣ک 20 da a9
#11 ن d9 86	**#12** \xd8\xa7\xd8 d8 a7 d8	**#13** ل d9 84	**#14** م d9 85	**#15** ں da ba	**#16** ␣م 20 d9 85	**#17** ␣ا 20 d8 a7	**#18** ت d8 aa	**#19** ␣ہ 20 db 81	**#20** یں db 8c da ba
#21 ھ da be	**#22** ␣س 20 d8 b3	**#23** د d8 af	**#24** س d8 b3	**#25** ␣ب 20 d8 a8	**#26** گ da af	**#27** ␣ن 20 d9 86	**#28** ␣پ 20 d9 be	**#29** ب d8 a8	**#30** ␣ت 20 d8 aa
#31 ور d9 88 d8 b1	**#32** ␣کی 20 da a9 db 8c	**#33** ان d8 a7 d9 86	**#34** ␣کے 20 da a9 db 92	**#35** \x20\xd8\xa7\xd8 20 d8 a7 d8	**#36** - db 94	**#37** ار d8 a7 d8 b1	**#38** ␣ج 20 d8 ac	**#39** ␣د 20 d8 af	**#40** ␣میں 20 d9 85 db 8c da ba
#41 ف d9 81	**#42** ق d9 82	**#43** ␣و 20 d9 88	**#44** ٹ d9 b9	**#45** چ da 86	**#46** ع d8 b9	**#47** ␣ہے 20 db 81 db 92	**#48** ال d8 a7 d9 84	**#49** وں d9 88 da ba	**#50** ␣کر 20 da a9 d8 b1

Token created at every 5,000th merge

Merge	ID	Token	Bytes
5,000	5,256	ضر	4
10,000	10,256	میمی	8
15,000	15,256	ۃ̃	6
20,000	20,256	␣البغ	9
25,000	25,256	␣کھنڈرات	15
30,000	30,256	␣مستعادہ	15
35,000	35,256	␣سہانے	11
40,000	40,256	␣وچھ	7
45,000	45,256	␣اشکالات	15

Early reconstruction

Letters rebuilt in the first 200 merges: ا #1 · ی #2 · ک #4 · ر #6 · و #7 · ہ #8 · ے #9 · ن #11 · ل #13 · م #14 · ں #15 · ت #18 · ھ #21 · د #23 · س #24 · گ #26 · ب #29 · ـ #36 · ف #41 · ق #42 · ٹ #44 · چ #45 · ع #46 · ج #53.

The first attested space-prefixed whole Urdu word is **␣کی** at merge 32 (ID 288). Earlier merges mainly reconstruct individual letters and space-plus-lead-byte fragments.

There are 106 learned tokens that are invalid UTF-8 alone. They are legal internal byte fragments and can be completed by later merges. Examples: #3 `20 d8` · #5 `20 d9` · #12 `d8 a7 d8` · #35 `20 d8 a7 d8` · #93 `e2 80` · #435 `20 e2 80`.

The progression is consistent with byte BPE. Frequent letter bytes are joined first, followed by common suffixes, function words and longer domain-specific strings. By merge 5,000, several regular Urdu words and names are already represented by single tokens.

3.1 Vocabulary Shape and Long Tokens

Twenty longest learned tokens

#	ID	Token	B	Chars
1	40886	▁آبادسندھپنجاببلوچستانخیبر	51	26
2	46806	صلی‌اللہ‌علیہ‌وآلہ‌وسلم	50	23
3	34898	اللہ‌علیہ‌وآلہ‌وسلم	44	20
4	40738	سندھپنجاببلوچستانخیبر	42	21
5	43776	▁----------	41	21
6	40826	▁بلتستاندنیابزنستعلیم	41	21
7	47163	▁ابراہیمعلیہ‌السلام	38	19
8	40782	▁پختونخواہکشمیرگلگت	37	19
9	34897	اللہ‌علیہ‌وآلہ	36	16
10	48828	▁خیبرپختونخواحکومت	35	18
11	48598	▁گورنرخیبرپختونخوا	35	18
12	28958	▁----------	35	18
13	40737	سندھپنجاببلوچستان	34	17
14	43184	▁ڈاکٹرطاہرالقادری	33	17
15	15691	----------	32	16
16	46769	▁خیبرپختونخوامیں	31	16
17	44079	▁اَلْمُؤْمِنِيْنَ	31	16
18	22131	BassersdorfZurichSwitzerlandP	30	30
19	49245	▁کاربوہائیڈریٹس	29	15
20	34779	اللہ‌علیہ‌وآ	29	13

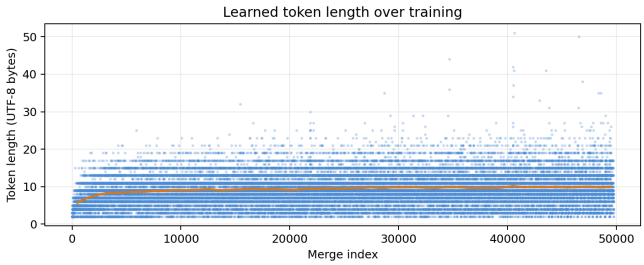


Figure 1. Learned token length across the 49,743 merge iterations.

Later merges combine recurring morphemes, space-prefixed words, names, punctuation and web-specific strings. Very long tokens also reveal residual boilerplate or unusual web text. For this reason, the longest-token table is useful both as a tokenizer result and as a corpus-quality check.

Implementation checks

- All 23 focused unit tests pass.
- Forty randomized reference and fast trainer checks pass.
- Empty strings, emoji, English, Urdu, newlines and the special marker round trip without loss.
- Maximum encoded token ID is 49,999, which fits `uint16`.

4. Question 2: Comparison with Existing Tokenizers

The held-out test split contains 743,572 words and 5.92 MiB after removing document markers. GPT-2 and `o200k_base` are used only for comparison, never for training or encoding our corpus.

Tokenizer	Total tokens	Tokens/word	Bytes/token
ours	882,054	1.186	7.040
gpt2	4,443,678	5.976	1.397
o200k_base	1,214,076	1.633	5.114

Our tokenizer uses 5.04 times fewer tokens than GPT-2 and 1.38 times fewer than `o200k_base` on this test set. The result reflects specialization to the training domains rather than a general ranking across languages.

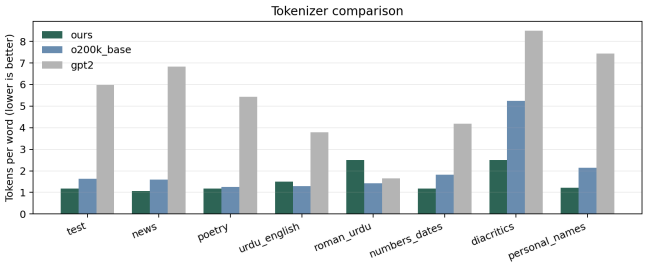


Figure 2. Tokens per word on the test split and seven required input types; lower is better.

4.1 Results on the Seven Required Inputs

Input type	Words	Ours	o200k_base	GPT-2
news	17	1.06	1.59	6.82
poetry	23	1.17	1.26	5.43
urdu english	14	1.50	1.29	3.79
roman urdu	14	2.50	1.43	1.64
numbers dates	16	1.19	1.81	4.19
diacritics	8	2.50	5.25	8.50
personal names	14	1.21	2.14	7.43

News, poetry, dates and personal names benefit from the Urdu-focused vocabulary. Roman Urdu is less efficient because abbreviated Latin chat spellings are rare in the corpus. Fully vowelled Arabic-style text also needs more pieces because its combining-mark sequences are sparse.

5. Question 3: Inputs the Tokenizer Handles Badly

Case 1: Roman Urdu with shortened spellings

???yr plzz mjhy bta do assignment kb submit krni hy

Ours: 2.50 tokens/word; o200k_base: 1.50; GPT-2: 1.70. Training is overwhelmingly Perso-Arabic, so shortened Latin spellings and repeated punctuation fall back to small byte pieces.

Pieces: yr | ▁pl | z | z | ▁m | j | hy | ▁b | ta | ▁do | ▁as | s

Case 2: Heavily vowelled Arabic-style text

قُرْآنِ حَكِيْمْ كِى تِلَاوَتْ بِالتَّجْوِيْدِ

Ours: 3.80 tokens/word; o200k_base: 6.80; GPT-2: 11.00. The regex correctly keeps marks with the word, but rare fully-vowelled combinations did not occur often enough to earn long merges.

Pieces: قُ | رْآن | ▁حَ | كِيْ | مْ | ▁كِ | ى | ▁تِ | لَ | او | تْ

Case 3: Mixed technical identifier

ماڈل XQ-97_v2.4 کا ID SHA256=9f86d081884c7d65 ہے۔

Ours: 1.45 tokens/word; o200k_base: 1.40; GPT-2: 2.00. Mixed identifiers are intentionally lossless, yet uncommon letter-digit-symbol sequences remain fragmented.

Pieces: ماڈ | ▁ID | ▁X | Q | - | 97 | _ | v | 2 | . | 4 | ▁کا

6. Deliverables and Reproduction

The repository includes the normalizer, both BPE trainers, collection and cleaning scripts, deterministic splitter, streaming encoder, tests, `tokenizer.json`, data card, analysis JSON and this report. The large-file folder should contain `train.txt`, `val.txt`, `test.txt`, `train.bin` and `val.bin`.

```
python -m unittest discover -s tests -v
python train_tokenizer.py --input corpus/train.txt --vocab-size 50000
python encode_corpus.py --tokenizer tokenizer.json
python scripts/analyze_tokenizer.py
python scripts/generate_report_pdf.py
```